

Comprehensive Architecture Roadmap & Implementation Plan: HRM System (MVP)

This document provides an in-depth codebase status analysis, database schema specification, and a step-by-step multi-phase implementation roadmap for building the **Human Resource Management (HRM) System MVP** for Royal Safari Tours.

1. Current Codebase & Architecture Analysis

Backend (backend/)

- **Framework & Language:** Node.js v20+, Express v5, TypeScript (`src/server.ts` , `src/app.ts`).
- **ORM & Database:** Prisma ORM (`@prisma/client v7.9.1`) connected to PostgreSQL / Supabase via `DATABASE_URL` and `DIRECT_URL` .
- **Modular Structure:** Located in `src/app/modules/` :
 - `auth/` : User authentication, JWT tokens, login/registration controllers & validations.
 - `user/` : User CRUD operations.
- **Middleware Infrastructure:** `src/app/middlewares/` containing authentication and role-based access control (RBAC).
- **File Uploads:** `multer` configured for document and photo handling.

Frontend (frontend/)

- **Framework:** Next.js 14 (App Router), React 18, TailwindCSS.
- **Dashboard Structure:** Located under `src/app/(dashboard)/dashboard/` :
 - Sidebar navigation and layout (`layout.js` , sidebar components).
 - Admin management modules (Users, Tour Packages, Testimonials, Analytics).
- **UI Components & Utilities:** Form validation with `react-hook-form` + `yup` , icon systems (`@iconify/react` , `lucide-react`), notifications (`react-hot-toast`), and Framer Motion animations.

2. Detailed Database Schema Design (prisma/schema.prisma)

The target schema isolates the **HRM MVP Core** while leaving dynamic relations and role enums ready for seamless future extension (Attendance, Leave, Payroll, Assets, Training).

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  directUrl = env("DIRECT_URL")
}

generator client {
  provider = "prisma-client-js"
}

// User role classifications for fine-grained RBAC
enum Role {
  SUPER_ADMIN
  ADMIN
  HR_MANAGER
  EMPLOYEE
  USER
}

// Central User model for credentials & dashboard log-in
model User {
  id      String   @id @default(uuid())
  name    String
  email   String   @unique
  avatar  String?
  role    Role      @default(USER)
  password String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  // Link: A user optionally has an Employee profile if they are part of staff
  employee Employee?
}

// Department Master Model
model Department {
  id      String   @id @default(uuid())
  name    String   @unique
  description String?
  employees Employee[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

// Designation Master Model
model Designation {
  id      String   @id @default(uuid())
```

```

name      String      @unique
description String?
employees Employee[]
createdAt DateTime    @default(now())
updatedAt DateTime    @updatedAt
}

// Employment Type Master Model (e.g. Full-Time, Part-Time, Contract, Intern)
model EmploymentType {
  id      String      @id @default(uuid())
  name    String      @unique
  employees Employee[]
  createdAt DateTime  @default(now())
  updatedAt DateTime  @updatedAt
}

// Employment Status Master Model (e.g. Active, Probation, Inactive, Suspended)
model EmploymentStatus {
  id      String      @id @default(uuid())
  name    String      @unique
  employees Employee[]
  createdAt DateTime  @default(now())
  updatedAt DateTime  @updatedAt
}

// Employee Profile model (HRM core data)
model Employee {
  id      String      @id @default(uuid())
  userId   String?     @unique // Optional connection to User for system access
  user     User?       @relation(fields: [userId], references: [id], onDelete: SetNull)
  employeeId String      @unique // Generated staff ID, e.g. "EMP-2026-0001"
  photo    String?     // URL to uploaded staff photo
  name     String
  email    String      @unique // Work or personal email
  phone    String?

  // Master references
  departmentId String
  department   Department @relation(fields: [departmentId], references: [id])
  designationId String
  designation   Designation @relation(fields: [designationId], references: [id])
  employmentTypeId String
  employmentType EmploymentType @relation(fields: [employmentTypeId], references: [id])
  employmentStatusId String
  employmentStatus EmploymentStatus @relation(fields: [employmentStatusId], references: [id])

  joiningDate DateTime @default(now())

  // Self-relation: manager assignment
  managerId String?
  manager   Employee? @relation("EmployeeManager", fields: [managerId], references: [id])
}

```

```

subordinates      Employee[]      @relation("EmployeeManager")

// HR notes (internal use, hidden from the Employee view)
hrNotes           String?

// Relationships
documents         EmployeeDocument[]

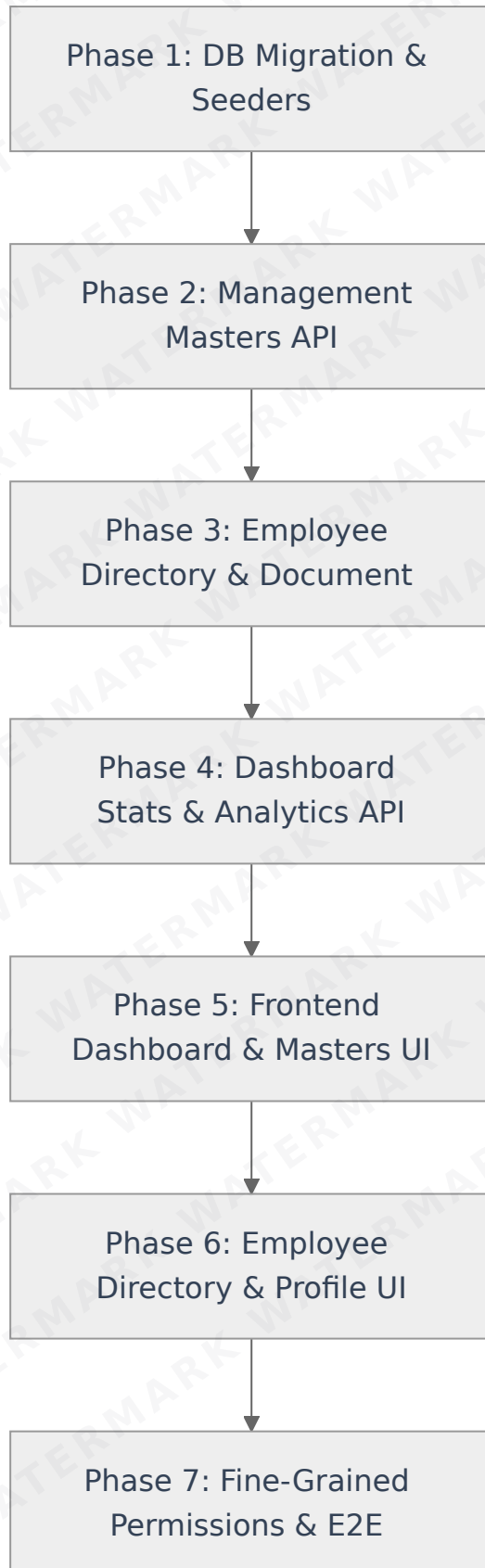
createdAt         DateTime         @default(now())
updatedAt         DateTime         @updatedAt
}

// Employee Document Library model
model EmployeeDocument {
  id              String           @id @default(uuid())
  employeeId      String
  employee        Employee @relation(fields: [employeeId], references: [id], onDelete: Cascade)
  documentName    String           // e.g. "NID_Copy.pdf", "Passport.pdf"
  fileUrl         String           // Path/URL to document file
  fileType        String?         // MIME type
  uploadedAt      DateTime         @default(now())
  updatedAt       DateTime         @updatedAt
}

```



3. Step-by-Step Multi-Phase Implementation Roadmap



Phase 1: Database Migration & Master Seeders (Backend)

1. **Schema Update:** Apply updated `schema.prisma` in `backend/prisma/schema.prisma`.
2. **Prisma Migration:** Generate migration and push to PostgreSQL/Supabase database (`npm run prisma migrate dev --name init_hrm_mvp`).

3. **Database Seeder:** Create `backend/prisma/seed.ts` to insert initial defaults:

- **Departments:** *Executive, Human Resources, Finance & Accounts, Operations & Safaris, Marketing & Sales, Customer Support, Information Technology.*
- **Designations:** *Managing Director, HR Manager, Senior Safari Guide, Accountant, Travel Consultant, Software Engineer, Support Executive.*
- **Employment Types:** *Full-Time, Part-Time, Contractual, Internship, Trainee.*
- **Employment Statuses:** *Active, Probation, Inactive, Suspended, On Leave.*

Phase 2: Management Masters Backend API (`backend/src/app/modules/`)

Create master modules for CRUD management:

- `department/` (`department.controller.ts`, `department.service.ts`, `department.route.ts`, `department.validation.ts`)
- `designation/` (`designation.controller.ts`, `designation.service.ts`, `designation.route.ts`, `designation.validation.ts`)
- `employment-type/` (`employmentType.controller.ts`, `employmentType.service.ts`, `employmentType.route.ts`)
- `employment-status/` (`employmentStatus.controller.ts`, `employmentStatus.service.ts`, `employmentStatus.route.ts`)

Phase 3: Employee Directory & Document Management API (Backend)

Create `employee/` and `employee-document/` backend modules:

1. **Auto Employee ID Generator:** Service helper `EMP-2026-0001` based on total annual registrations.

2. **Employee Endpoints:**

- `GET /api/v1/employees` (Search, Filter by Department/Designation/Status/Type, Pagination, Sorting).
- `POST /api/v1/employees` (Create employee record + optional linked `User` credentials).
- `GET /api/v1/employees/:id` (Fetch detailed profile with documents & manager info).
- `PATCH /api/v1/employees/:id` (Update employee details, status, or manager).
- `DELETE /api/v1/employees/:id` (Deactivate/delete employee).
- `GET /api/v1/employees/me` (Employee self-profile endpoint).

3. **Employee Document Library Endpoints:**

- `POST /api/v1/employees/:id/documents` (Multer upload for NID, Passport, Certificates).
- `DELETE /api/v1/employees/documents/:docId` (Remove document).

Phase 4: Basic Dashboard Stats API (Backend)

Create `hrm-dashboard/` module:

- GET /api/v1/hrm/dashboard/stats : Returns JSON:

- totalEmployees
- activeEmployees
- probationEmployees
- inactiveEmployees
- departmentBreakdown (count per department)
- recentJoins (last 5 newly joined staff)

Phase 5: Frontend Dashboard & Management Masters UI

(frontend/src/app/(dashboard)/dashboard/)

1. **Sidebar Navigation Update:** Add **Human Resources** section in dashboard sidebar:

-  HRM Overview (/dashboard/hrm)
-  Employee Directory (/dashboard/hrm/employees)
-  Document Library (/dashboard/hrm/documents)
-  Masters Setup (/dashboard/hrm/masters - Departments, Designations, Types, Statuses)

2. **Management Masters Pages:**

- Interactive CRUD modal tables for Departments, Designations, Employment Types, and Employment Statuses.

Phase 6: Employee Directory, Form & Profile UI (Frontend)

1. **HRM Overview Dashboard Page** (/dashboard/hrm/page.jsx):

- Metric cards (Total Staff, Active, Probation, Inactive).
- Department distribution charts/bars.
- Quick action buttons (Add New Employee, Manage Masters).

2. **Employee Directory Page** (/dashboard/hrm/employees/page.jsx):

- Search bar (by name, email, staff ID).
- Filter dropdowns (Department, Designation, Status, Type).
- Action dropdowns (View Profile, Edit Details, Activate/Deactivate, Delete).

3. **Employee Add/Edit Modal/Page** (/dashboard/hrm/employees/create/page.jsx):

- Form fields: Name, Email, Phone, Photo Upload, Department, Designation, Type, Status, Manager, Joining Date, HR Notes.

4. **Employee Profile View Page** (/dashboard/hrm/employees/[id]/page.jsx):

- **Tab 1: Personal & Contact Info**
- **Tab 2: Employment Details & Manager**
- **Tab 3: Document Library** (File upload, preview, download, delete)
- **Tab 4: Internal HR Notes** (Visible to Admin/HR only)

Phase 7: Fine-Grained Permissions & Employee Self-Service (RBAC + Future Proofing)

1. RBAC Guarding:

- SUPER_ADMIN & ADMIN : Full access to create, edit, delete, assign manager/status, and view HR notes.
- HR_MANAGER : Create/edit employee data, manage documents, update statuses.
- EMPLOYEE : Restrict view strictly to own profile (/dashboard/hrm/my-profile), update phone/contact info, change password.

2. Future Expansion Architectural Readiness:

- Schema and modules ready for Phase 2 expansions: **Attendance, Leave Requests, Payroll, Recruitment & Onboarding, Asset Tracking, and Performance Appraisals.**



Proposed File Structure Changes

```
backend/
├── prisma/
│   ├── schema.prisma [MODIFY] (Apply new HRM Postgres/Supabase schema)
│   └── seed.ts [NEW] (Seed initial departments, designations, types, status)
└── src/
    └── app/
        └── modules/
            ├── department/ [NEW] (Department CRUD API)
            ├── designation/ [NEW] (Designation CRUD API)
            ├── employmentType/ [NEW] (Employment Type CRUD API)
            ├── employmentStatus/ [NEW] (Employment Status CRUD API)
            ├── employee/ [NEW] (Employee Directory & Profile CRUD API)
            ├── employeeDocument/ [NEW] (Document Uploads API)
            └── hrmDashboard/ [NEW] (HRM Analytics & Stats API)

frontend/
├── src/
│   └── app/
│       └── (dashboard)/
│           └── dashboard/
│               └── hrm/ [NEW]
│                   ├── page.jsx (HRM Overview & Metrics)
│                   ├── masters/page.jsx (Departments, Designations, Types, Statuses)
│                   ├── employees/
│                       ├── page.jsx (Employee Directory Table & Filters)
│                       ├── create/page.jsx (Create New Staff Form)
│                       └── [id]/
│                           └── page.jsx (Employee Profile, Tabs & Document Library)
```


Verification & Quality Assurance Plan

Backend Verification

- Execute `npx prisma db push` OR `npx prisma migrate dev` to verify clean schema generation.
- Run `npm run dev` in `backend/` to verify zero TypeScript errors.
- Test CRUD endpoints using REST Client or Postman.

Frontend Verification

- Execute `npm run build` in `frontend/` to verify compilation across all new dashboard routes.
- Verify responsive layout across Desktop, Tablet, and Mobile screens.

